Introduction

Low-Level Design (LLD) interviews evaluate your ability to transform real-world scenarios into clean, scalable, and easy-to-maintain code structures. One of the most commonly discussed examples is the **Ride Booking App**.

In this article, we'll guide you through a clear step-by-step framework to design this system, just like you would in a real interview. Along the way, we'll share pro tips to help you articulate your reasoning, handle edge cases, and impress your interviewer with a well-thought-out approach.

Step 1: Clarify Requirements

Functional Requirements:

- As a rider, I can register and log in.
- As a driver, I can register/onboard and go online or offline.
- As a rider, I can set pickup and destination using map or search.
- As a rider, I can view an upfront fare estimate along with ETA before requesting a ride.
- As a rider, I can request a ride based on the shown fare estimate.
- As a rider, I can cancel a ride before pickup, subject to cancellation policy.
- The system asynchronously matches a rider to the nearest available driver from all online drivers (top N by distance).
- The system ensures a driver is not assigned to multiple active rides simultaneously.
- As a driver, I can accept or decline a ride request within a timeout window.
- As a driver, I can navigate to the pickup location and then to the drop-off location.

- As a driver, I can start and complete a trip.
- As a rider, I can track the driver's real-time location and trip progress using GPS.
- The driver app sends location updates to the system every N seconds.
- Payment is processed upon trip completion using supported methods (card/wallet/cash).
- As a rider, I receive a payment receipt after trip completion.
- Cancellation fees are applied based on defined policy rules.

Interview Tip: *Clearly validating end-to-end user flows (rider + driver) before designing the system demonstrates strong product and system thinking.*

Counter Questions to Ask Interviewer

- What is the driver matching strategy: nearest distance or fastest ETA?
- What is the search radius and ride request accept-timeout for drivers?
- How is fare calculated (base + distance + time)? Is surge pricing required?
- How is payment handled: pre-authorization or post-trip charge?
- Is pricing fixed upfront or recalculated after trip completion?
- How does the cancellation policy work, especially after payment initiation?

Step 2: Identify Core Entities

1. Ride (Core Entity)

- id: int [PK]
- rideId: String [UNIQUE]
- riderId: int [FK to Rider]
- driverId: int [FK to Driver, NULLABLE] (NULL until assigned)
- pickupLocation: Location (latitude, longitude, address)
- dropoffLocation: Location (latitude, longitude, address)
- status: RideStatus (Enum: REQUESTED, ASSIGNED, ACCEPTED, IN_PROGRESS, COMPLETED, CANCELLED)
- estimatedFare: long (Stored as integer: actual amount * 100)
- estimatedDistance: double (in km)
- actualDistance: double [NULLABLE] (used for analytics/ETA tuning)
- estimatedDuration: long (in seconds)
- actualDuration: long [NULLABLE] (used for analytics/ETA tuning)
- requestedAt: LocalDateTime
- assignedAt: LocalDateTime [NULLABLE]
- acceptedAt: LocalDateTime [NULLABLE]
- startedAt: LocalDateTime [NULLABLE]
- completedAt: LocalDateTime [NULLABLE]
- cancelledAt: LocalDateTime [NULLABLE]
- cancellationReason: String [NULLABLE]
- paymentType: PaymentType (Enum: PRE_PAYMENT, POST_PAYMENT)
- paymentId: String [NULLABLE] (Payment gateway transaction ID)
- paymentStatus: PaymentStatus (Enum: PENDING, COMPLETED, FAILED, REFUNDED)

2. Rider

- id: int [PK]
- username: String [UNIQUE]
- email: String [UNIQUE]

- phoneNumber: String [UNIQUE]
- name: String
- createdAt: LocalDateTime

3. Driver

- id: int [PK]
- username: String [UNIQUE]
- email: String [UNIQUE]
- phoneNumber: String [UNIQUE]
- name: String
- licenseNumber: String [UNIQUE]
- vehicleNumber: String [UNIQUE]
- vehicleType: String (Sedan, SUV, etc.)
- isOnline: boolean (true when driver is available)
- currentLocation: Location [NULLABLE] (updated via GPS)
- lastLocationUpdate: LocalDateTime [NULLABLE]
- createdAt: LocalDateTime

4. Location

- latitude: double
- longitude: double
- address: String [NULLABLE]
- timestamp: LocalDateTime

5. RideStatus (Enum)

- REQUESTED
- ASSIGNED
- ACCEPTED
- IN_PROGRESS
- COMPLETED
- CANCELLED

6. PaymentStatus (Enum)

- PENDING
- COMPLETED
- FAILED
- REFUNDED

7. PaymentType (Enum)

- PRE_PAYMENT
- POST_PAYMENT

8. DriverStatus (Enum)

- ONLINE
- OFFLINE
- ON_RIDE

9. RideStatusResponse (Response DTO - Not Stored)

- rideId: String
- status: RideStatus
- driver: DriverInfo [NULLABLE] (id, name, phoneNumber, vehicleNumber, currentLocation, eta)
- currentLocation: Location [NULLABLE]
- estimatedFare: long [NULLABLE]
- pickupLocation: Location
- dropoffLocation: Location
- requestedAt: LocalDateTime
- Note: Built dynamically for ride-status polling.

10. FareEstimateResponse (Response DTO)

- estimatedFare: long (actual amount * 100)
- estimatedDistance: double (in km)
- estimatedDuration: long (in seconds)
- currency: String (e.g., "USD")

11. RideRequest (Request DTO)

- riderId: int
- pickupLocation: Location
- dropoffLocation: Location
- paymentType: PaymentType
- Note: POST_PAYMENT is cash-only; PRE_PAYMENT uses payment gateway.

12. FareEstimateRequest (Request DTO)

- pickupLocation: Location
- dropoffLocation: Location

13. LocationUpdateRequest (Request DTO)

- driverId: int
- location: Location
- timestamp: LocalDateTime

Interview Tip: *Clearly separating core entities from DTOs shows strong system-design maturity and helps scale matching, tracking, and payments independently.*

Step 3: Visualize Interaction Flows

1. Fare Estimate Flow

- GET /api/rides/fare-estimate → RideController.getFareEstimate(request)
- → PricingService.calculateFare(pickup, dropoff)
- → MapService.getDistanceAndDuration(pickup, dropoff)
- → PricingStrategy.calculateFare(distance, duration)
- → Return FareEstimateResponse

2. Ride Request Flow

2.1 Request Ride (Initial)

- POST /api/rides/request → RideController.requestRide(request)
- → RideService.requestRide(request)
- → Validate rider exists
- → Calculate estimated fare
- → Create Ride(status=REQUESTED, paymentType, estimatedFare)

2.2 PRE_PAYMENT Ride Flow

- → PaymentService.initiatePayment(rideId, estimatedFare)
- → PaymentGatewayRouter.selectProvider(...) → provider.initiatePayment(...)
- → Update Ride(paymentStatus=PENDING, paymentId)
- → Do NOT start driver matching yet
- → Return {rideId, status: REQUESTED, paymentStatus: PENDING}

2.3 POST_PAYMENT Ride Flow (Cash)

- → Validate cash-only payment
- → Start Async MatchingService.matchDriver(ride)
- → Return {rideId, status: REQUESTED}

2.4 Payment Callback (PRE_PAYMENT Only)

- POST /api/payments/callback → PaymentController.handlePaymentCallback(transactionId, status)
- → Verify callback authenticity
- → Fetch ride by paymentId
- → On SUCCESS: Update Ride(paymentStatus=COMPLETED) → Start matching → Notify rider
- → On FAILURE: Update Ride(status=CANCELLED, paymentStatus=FAILED) → Notify rider

3. Async Driver Matching Flow

- MatchingService.matchDriver(ride)
- → Ensure ride status is REQUESTED

- → Fetch available drivers (DriverStatus.ONLINE)
- → Apply MatchingStrategy (nearest/top-N drivers)
- → For each driver candidate:
    - Acquire distributed lock on driver
    - Validate driver still ONLINE
    - Push ride notification to driver
    - Wait for response (poll ride status, timeout 30s)
    - If ACCEPTED → stop matching
    - If timeout / decline → try next driver
    - Release driver lock
- → If no driver accepts → return empty

4. Driver Accept / Decline Flow

4.1 Driver Accept Ride

- POST /api/rides/{rideId}/accept → RideController.acceptRide
- → Acquire ride-level lock
- → If status REQUESTED: assign driver (status=ASSIGNED)
- → Set driver status ON_RIDE
- → Update Ride(status=ACCEPTED)
- → Notify rider

4.2 Driver Decline Ride

- POST /api/rides/{rideId}/decline → RideController.declineRide
- → Acquire ride-level lock
- → If REQUESTED: ignore (no assignment yet)
- → If ASSIGNED: release driver → reset ride → re-trigger matching

5. Rider Polling for Ride Status

- GET /api/rides/{rideId}/status → RideController.getRideStatus
- → RideService.getRideStatus
- → Build RideStatusResponse
- → Rider polls every 2–3 seconds

6. Driver Location Updates (GPS)

- POST /api/drivers/{driverId}/location → DriverController.updateLocation
- → Update Driver.currentLocation & lastLocationUpdate
- → If active ride exists: push location update to rider

7. Start Trip

- POST /api/rides/{rideId}/start → RideController.startRide
- → Validate ride is ACCEPTED
- → Update Ride(status=IN_PROGRESS, startedAt)
- → Notify rider & enable real-time tracking

8. Complete Trip

- POST /api/rides/{rideId}/complete → RideController.completeRide
- → Validate ride is IN_PROGRESS
- → Capture actual distance & duration (telemetry)
- → Update Ride(status=COMPLETED)
- → Handle payment completion (cash or prepaid)
- → Notify rider & release driver

9. Cancel Ride

- POST /api/rides/{rideId}/cancel → RideController.cancelRide
- → Validate cancellation policy
- → Update Ride(status=CANCELLED, reason)
- → Release driver if assigned
- → Apply cancellation fee if applicable
- → Notify rider and driver

10. Driver Online / Offline

- POST /api/drivers/{driverId}/online → DriverService.goOnline
- → Set DriverStatus.ONLINE & register for notifications
- POST /api/drivers/{driverId}/offline → DriverService.goOffline
- → Set DriverStatus.OFFLINE & unregister notifications

Step 4: Define Class Structures & Relationships

Layers of Architecture

We will design the **architecture layers** of the system in a structured way, ensuring **separation of concerns** and **modularity**. The system will be organized into the following layers:

Client/UI → Controller Layer → Service Layer → Domain Layer

CONTROLLERS

1. RideController

- - RideStatusResponse getRideStatus(String rideId)
- - Ride requestRide(RideRequest request)
- - void cancelRide(String rideId, String reason)
- - FareEstimateResponse getFareEstimate(FareEstimateRequest request)

2. DriverController

- - void acceptRide(String rideId, String driverId)
- - void declineRide(String rideId, String driverId)
- - void startRide(String rideId, String driverId)
- - void completeRide(String rideId, String driverId)
- - void updateLocation(String driverId, Location location)
- - void goOnline(String driverId)
- - void goOffline(String driverId)

3. PaymentController

- - void handlePaymentCallback(String transactionId, PaymentStatus status)

SERVICES

1. RideService

- - Ride requestRide(RideRequest request)
- - RideStatusResponse getRideStatus(String rideId)
- - void cancelRide(String rideId, String reason)
- - void driverAccept(String rideId, String driverId)
- - void driverDecline(String rideId, String driverId)
- - void startRide(String rideId, String driverId)
- - void completeRide(String rideId, String driverId)

2. MatchingService (Async)

- - Optional<Driver> matchDriver(Ride ride)
- - void releaseDriver(String driverId)

3. PricingService

- - FareEstimateResponse calculateFare(Location pickup, Location dropoff)

4. PaymentService

- - String initiatePayment(String rideId, long amount)
- - void handlePaymentCallback(String transactionId, PaymentStatus status)

5. LocationService

- - void updateDriverLocation(int driverId, Location location)
- - Location getDriverLocation(int driverId)
- - double calculateDistance(Location loc1, Location loc2)

- - long calculateETA(Location from, Location to)

6. DriverService

- - void goOnline(int driverId)
- - void goOffline(int driverId)
- - Driver getById(int driverId)
- - boolean isAvailable(int driverId)

7. LockService (Distributed)

- - boolean acquire(String key, long timeoutMs)
- - void release(String key)

8. NotificationService

- - void sendToDriver(int driverId, NotificationMessage message)
- - void sendToRider(int riderId, NotificationMessage message)

9. MapService (External Integration)

- - DistanceAndDuration getDistanceAndDuration(Location from, Location to)
- - String geocode(Location location)
- - Location reverseGeocode(double lat, double lon)

RIDE STATE PATTERN (State Machine)

- **RideState (Interface)**
  - + void accept(Ride ride, int driverId)
  - + void cancel(Ride ride, int userId, String reason)
  - + void start(Ride ride, int driverId)
  - + void complete(Ride ride, int driverId)
- RequestedState
- AssignedState
- AcceptedState

- InProgressState
- CompletedState
- CancelledState

Note: State objects are created dynamically from RideStatus enum; only enum is persisted.

DRIVER MATCHING STRATEGY

- **DriverMatchingStrategy (Interface)**
    - + List<Driver> findMatchingDrivers(Location pickup, List<Driver> candidates, int maxResults)
- NearestDriverStrategy
- FastestEtaStrategy (Future)
- **MatchingService** acts as strategy context

PRICING STRATEGY

- **PricingStrategy (Interface)**
    - + long calculateFare(double distance, long duration, PricingContext context)
- BasePricingStrategy
- SurgePricingStrategy
- **PricingService** acts as strategy context

PAYMENT GATEWAY STRATEGY

- **PaymentGatewayProvider (Interface)**
    - + String getName()
    - + String initiatePayment(String rideId, long amount, Map<String, String> paymentDetails)
    - + boolean verifyCallback(String transactionId, PaymentStatus status)
- StripePaymentGatewayProvider
- RazorpayPaymentGatewayProvider
- PayPalPaymentGatewayProvider
- MockPaymentGatewayProvider
- **PaymentGatewayRouter** selects provider dynamically

REPOSITORIES

1. RideRepository

- - Optional<Ride> findByRideId(String rideId)
- - Ride save(Ride ride)
- - List<Ride> findByRiderId(int riderId)
- - List<Ride> findByDriverId(int driverId)
- - List<Ride> findByStatus(RideStatus status)

2. RiderRepository

- - Optional<Rider> findById(int id)
- - Optional<Rider> findByEmail(String email)
- - Rider save(Rider rider)

3. DriverRepository

- - Optional<Driver> findById(String id)
- - List<Driver> findByStatus(DriverStatus status)
- - Driver save(Driver driver)
- - void updateLocation(String driverId, Location location)

4. LocationRepository

- - void saveLocation(int driverId, Location location)
- - Location getLatestLocation(int driverId)

KEY RELATIONSHIPS & PATTERNS

- **Association** – Ride references Rider and Driver
- **Aggregation** – Ride contains pickup and dropoff Location
- **Dependency** – Services depend on repositories and external services
- **State Pattern** – Ride lifecycle handling
- **Strategy Pattern** – Pricing, Matching, Payment Gateway
- **Repository Pattern** – Data persistence abstraction

Step 5: Core Use Cases & Methods

1. Request Ride Use Case:

requestRide(request) →

- Validate: rider exists, pickup ≠ dropoff, valid locations
- Calculate estimated fare
- Create Ride(status=REQUESTED, requestedAt=now, paymentType, estimatedFare)
- Lock estimatedFare as the final agreed price (no adjustments after trip)
- If paymentType is **PRE_PAYMENT**:
    - Initiate payment via PaymentService.initiatePayment(rideId, estimatedFare)

    - PaymentGatewayRouter.selectProvider(...) → provider.initiatePayment(...)
    - Update Ride(paymentStatus=PENDING, paymentId=transactionId)
    - Do NOT start matching yet (wait for payment callback)
    - Return {rideId, status: REQUESTED, paymentStatus: PENDING} immediately
- If paymentType is **POST_PAYMENT**:
    - Validate: only cash payment allowed
    - Trigger async MatchingService.matchDriver(ride) immediately
- Return {rideId, status: REQUESTED} immediately (non-blocking)

---

2. Match Driver Use Case (Async):

matchDriver(ride) →

- Fetch ride; ensure status is REQUESTED
- Find available drivers: DriverRepository.findByStatus(ONLINE)
- Apply MatchingStrategy.findMatchingDrivers(pickup, drivers, maxResults=3)
- For each driver candidate (in order):
    - Acquire distributed lock on driver (timeout=200ms)
    - Re-fetch driver and validate still ONLINE
    - Keep driver ONLINE (not assigned yet)
    - Push ride notification to driver
    - Wait for response (poll every 500ms, timeout 30s):
        - If ACCEPTED by same driver → return driver
        - If CANCELLED → return empty
        - If another driver assigned → break
        - If timeout → continue to next driver
    - Release driver lock

- If no drivers accepted → return empty Optional

---

3. Accept Ride Use Case:

driverAccept(rideId, driverId) →

- Acquire distributed lock on ride
- Fetch ride
- If status is REQUESTED: assign driver (status=ASSIGNED, assignedAt=now)
- Validate driverId matches assigned driver
- Update driver status to ON_RIDE
- Update Ride(status=ACCEPTED, acceptedAt=now)
- Transition ride state via State Pattern
- Push notification to rider
- Release lock

---

4. Decline Ride Use Case:

driverDecline(rideId, driverId) →

- Acquire distributed lock on ride
- Fetch ride
- If status is REQUESTED: return (matching continues)
- If status is ASSIGNED:
    - Validate driverId
    - Release driver (set ONLINE)
    - Update Ride(status=REQUESTED, driverId=null)
    - Trigger re-matching
- Release lock

---

5. Start Trip Use Case:

startRide(rideId, driverId) →

- Acquire distributed lock on ride
- Validate status is ACCEPTED and driver matches
- Update Ride(status=IN_PROGRESS, startedAt=now)
- Transition ride state via State Pattern
- Start real-time location tracking
- Push notification to rider
- Release lock

---

6. Complete Trip Use Case:

completeRide(rideId, driverId) →

- Acquire distributed lock on ride
- Validate status is IN_PROGRESS and driver matches
- Capture actual distance & duration (analytics only)
- Update Ride(status=COMPLETED, completedAt=now)
- Transition ride state via State Pattern
- If POST_PAYMENT:
    - Driver collects locked estimatedFare (cash)
    - Update Ride(paymentStatus=COMPLETED)
- If PRE_PAYMENT:
    - Fare already paid via gateway
- Mark driver as available
- Push receipt notification
- Release lock

---

7. Payment Callback Use Case (PRE_PAYMENT):

handlePaymentCallback(transactionId, status) →

- Verify callback authenticity
- Find ride by paymentId
- If SUCCESS:
    - Update Ride(paymentStatus=COMPLETED)
    - Trigger async MatchingService.matchDriver(ride)
    - Notify rider
- If FAILURE:
    - Update Ride(status=CANCELLED, paymentStatus=FAILED)
    - Notify rider

---

8. Cancel Ride Use Case:

cancelRide(rideId, userId, reason) →

- Acquire distributed lock on ride
- Validate user is rider or driver
- Validate cancellation policy
- Update Ride(status=CANCELLED, cancelledAt=now)
- Release driver if assigned
- Apply cancellation fee if needed
- Push notifications
- Release lock

---

9. Update Driver Location Use Case:

updateDriverLocation(driverId, location) →

- Update driver current location
- If ride IN_PROGRESS:
    - Calculate ETA
    - Push live location to rider
- Persist location history

---

10. Get Ride Status Use Case (Polling):

getRideStatus(rideId) →

- Fetch ride
- Fetch driver & location if assigned
- Calculate ETA if IN_PROGRESS
- Build RideStatusResponse DTO
- Return response (polled every 2–3 seconds)

Step 6: Apply OOP Principles & Design Patterns

Design Patterns Used:

- **Repository Pattern –** data access abstraction for Ride, Driver, Rider, and Location
- **Service Layer –** separation of business logic (RideService, MatchingService, PricingService, PaymentService)
- **State Pattern –** ride lifecycle management (RequestedState, AssignedState, AcceptedState, InProgressState, CompletedState, CancelledState)

- **Strategy Pattern –** DriverMatchingStrategy (NearestDistanceStrategy, FastestEtaStrategy)
- **Strategy Pattern –** PricingStrategy (BasePricingStrategy, SurgePricingStrategy)
- **Strategy Pattern –** PaymentGatewayProvider with Router (Stripe, Razorpay, PayPal, Mock)
- **Observer Pattern (Lightweight) –** driver location updates trigger rider notifications
- **RESTful API Design –** clean, resource-oriented endpoints

---

OOP Principles Applied:

- **Single Responsibility –** each service handles one concern (RideService manages rides, MatchingService handles driver matching, etc.)
- **Open/Closed –** new matching strategies, pricing strategies, or payment gateways can be added without modifying core logic
- **Encapsulation –** ride state transitions occur only through the State Pattern; location updates only through LocationService
- **Dependency Inversion –** services depend on abstractions (repositories, strategies, payment gateways, map services)
- **Idempotency –** payment callbacks and ride actions are safe to retry without duplicate processing

---

Step 7: Handle Edge Cases

Edge Case Solutions:

- **No Drivers Available:**

    - If no drivers accept after trying all candidates, return empty (no match)
    - Optionally retry matching later or notify the rider
- **Driver Timeout (Doesn't Accept):**
    - Define accept-timeout (e.g., 30 seconds)
    - If timeout expires, move to the next best driver
- **Driver Declines Ride:**
    - Immediately re-match to the next best available driver
    - Track decline count to avoid repeatedly notifying the same driver
- **Multiple Concurrent Ride Requests from Same Rider:**
    - Ensure rider has no active ride (REQUESTED, ASSIGNED, ACCEPTED, IN_PROGRESS)
    - Reject new requests until the current ride completes or cancels
- **Driver Double-Assignment:**
    - Use distributed locks during matching and acceptance (STEP-7.5)
    - Ensure a driver can be assigned to only one active ride
- **Location Updates Failure:**
    - Treat driver location updates as best-effort
    - Fallback to last known location if updates fail
- **Payment Failure:**
    - Retry payment with exponential backoff
    - Notify rider and allow manual retry
- **Ride Cancellation After Payment:**
    - Trigger refund via payment gateway
    - Update ride and payment status accordingly
- **GPS Spoofing / Fake Locations:**
    - Validate that location updates are reasonable (no teleporting)
    - Rate-limit driver location updates
- **Network Failures During Ride:**

    - Clients retry requests with exponential backoff
    - Server maintains authoritative ride state and resumes on reconnect
- **Driver Goes Offline During Matching:**
    - Re-validate driver online status before assignment
    - If offline, skip and match the next driver
- **Invalid Pickup or Drop-off Locations:**
    - Validate coordinates fall within the supported service area
    - Reject requests outside service bounds
- **Duplicate Ride Requests (Retries):**
    - Use idempotency key for ride requests
    - Prevent duplicate rides with same pickup/dropoff in a short time window
- **Payment Gateway Unavailable:**
    - Queue payment request for retry
    - Notify rider and allow alternative payment method
- **Real-Time Tracking Staleness:**
    - Return last known driver location with timestamp
    - Client displays “last updated X seconds ago”

Step 7.5: Concurrency Handling Methods (Preventing Double-Assignment)

Overview:

Multiple concurrent requests trying to assign the same driver to different rides can cause double-assignment. Here are 3 generic approaches to handle this:

Approach 1: Database Transaction with Row-level Locking

Description:

- Use database transaction with high isolation level (SERIALIZABLE or REPEATABLE_READ)
- Acquire exclusive locks on driver and ride rows using SELECT FOR UPDATE
- Locks are held until transaction commits or rolls back

Example Flow:

- BEGIN TRANSACTION
- SELECT * FROM driver WHERE id = 1 AND isOnline = true FOR UPDATE
- SELECT * FROM ride WHERE id = 5 AND status = 'REQUESTED' FOR UPDATE
- Validate: driver.isOnline == true and driver has no active ride
- UPDATE ride SET driverId = 1, status = 'ASSIGNED' WHERE id = 5
- COMMIT TRANSACTION

Benefits:

- Database ensures atomicity (all or nothing)
- Prevents concurrent modifications
- Works with any SQL database
- Language-independent

Limitations:

- Locks held for entire transaction duration
- Deadlocks possible if locks are acquired in different order
- Reduced throughput under high concurrency

Approach 2: Optimistic Locking with Version Field

Description:

- Add version or timestamp field to Driver and Ride entities
- Read entities with their current version
- Verify version has not changed before update

- If version changed, retry or reject operation

Example Flow:

- READ driver (id=1, isOnline=true, version=5)
- READ ride (id=5, status=REQUESTED, version=3)
- Validate: driver.isOnline == true
- UPDATE ride SET driverId = 1, status = 'ASSIGNED', version = version + 1 WHERE id = 5 AND version = 3
- If update affects 0 rows → Version conflict → Retry or fail
- If update affects 1 row → Success

Benefits:

- No long-held locks
- Scales well for high-read systems
- Database and language agnostic
- Retry-based conflict resolution

Limitations:

- Requires retry logic
- May fail multiple times under contention
- More complex error handling

Approach 3: Distributed Locking (Selected Approach)

Description:

- Use distributed lock manager (Redis, etc.)
- Acquire lock on driver ID before assignment
- Release lock after assignment completes
- Lock expiration prevents deadlocks

Example Flow:

- ACQUIRE LOCK("driver_lock_1", timeout=200ms)
    - If lock acquired → Proceed

    - If lock exists → Skip and try next driver
    - If timeout expires → Continue matching
- READ driver status (isOnline, no active ride)
- Validate: driver is available
- UPDATE ride to assign driver
- RELEASE LOCK("driver_lock_1")

Benefits:

- Works across multiple application servers
- Prevents double-assignment at application level
- Low lock duration → high throughput
- Deadlock-safe via lock expiration
- Well-suited for real-time matching systems

Limitations:

- Requires external infrastructure (Redis)
- Network latency for lock operations
- Must handle lock expiration carefully

Selected Approach: Distributed Locking (Approach 3)

The system uses **distributed locking** to prevent driver double-assignment:

- **Scalability:** Works across multiple application servers
- **Flexibility:** Database and language agnostic
- **Reliability:** Lock expiration avoids deadlocks
- **Performance:** Non-blocking locks enable fast matching
- **Interview-friendly:** Demonstrates real-world distributed system design

Implementation Details:

- **Lock Service:** Redis or similar distributed cache

- **Driver Lock Key:** "driver_lock_{driverId}"
- **Ride Lock Key:** "ride_lock_{rideId}"
- **Lock Timeout:** 200ms (driver), 500ms (ride)
- **Lock Acquisition:** Non-blocking; skip on timeout
- **Lock Ordering:** Acquire locks in sorted order (driver → ride)
- **Lock Release:** Always release in finally block

Step 9: Future Improvements

Future Functional Requirements:

- Post-trip ratings and reviews (rider rates driver, driver rates rider)
- Advance ride scheduling (book rides for future time)
- Multiple ride categories (X, XL, Comfort, Premium) with different pricing
- Shared/pooled rides with multiple riders
- Surge pricing based on demand and supply
- Promo codes and discounts
- In-app chat/call between rider and driver
- Multi-stop trips (pickup multiple passengers or multiple destinations)
- Driver earnings dashboard and payout management
- Ride history and receipts export (CSV/PDF)
- Real-time traffic integration for better ETA calculations
- Geofencing for pickup/dropoff verification
- Safety features (SOS button, emergency contacts, trip sharing)
- Driver incentives and bonuses
- Integration with public transit for multi-modal trips

Summary

This design targets a 60–120 minute interview with 5 core flows:

- **Fare Estimate:** GET /api/rides/fare-estimate (distance, duration, pricing)
- **Request Ride:** POST /api/rides/request (async matching, non-blocking response)
- **Driver Matching:** Async assignment with push notifications to driver
- **Ride Lifecycle:** Accept / Start / Complete with state transitions and real-time tracking
- **Payment Processing:** External gateway integration with callback handling

---

Highlights:

- State Pattern for ride lifecycle management
- Strategy Pattern for driver matching, pricing, and payment gateways
- Distributed locking to prevent double-assignment of drivers (STEP-7.5)
- Upfront fare estimate locked as the single payable price (no final-price adjustments)
- Hybrid communication: Rider polling + push notifications; Driver push notifications (primary)
- Real-time location tracking via GPS updates and polling endpoint
- External payment gateway integration (no internal payment processing)
- Repository/Service layering with clear separation of concerns
- Idempotent operations for safe retries
- Clear edge-case coverage suitable for production conversations